SECURITY ASSESSMENT

The "Caveman" Claude Code Skill

Third-party skill review — implications for use alongside Claude Code in highly confidential / enterprise environments

Prepared: July 23, 2026

---

1. Executive Summary

"Caveman" is not an Anthropic product. It is a community-authored, MIT-licensed skill/plugin (maintainer: JuliusBrussee, GitHub) that rewrites Claude's spoken output into a terse, compressed register — dropping filler words and articles — to cut output tokens, while its own rules instruct it to leave code, commands, and error text untouched. It installs into Claude Code and 30+ other coding agents via a single shell installer.

The core finding of this review: the terse-writing prompt itself adds little direct security risk. Nearly all of the additional risk for a confidential enterprise environment comes from how the project is distributed and installed by default — an unpinned curl-to-bash installer, a second chained remote script, an MCP proxy that is registered by default and sits between Claude Code and other data sources, and a local stats reader that parses session logs. There is also a real but softer risk that terse output strips exactly the narrative detail (impact, rationale, audit trail) that compliance-driven review processes expect.

Three tests were performed: a static security review of the published installer, and two rule-application simulations that apply the skill's own documented ruleset to a destructive-operation scenario and a security code-review scenario. Results and a risk matrix are below, followed by concrete recommendations.

2. What the Caveman Skill Actually Is

2.1 Mechanism

At its core, caveman is a SKILL.md file — plain-text behavioral instructions loaded into Claude's context, the same mechanism as any custom skill or CLAUDE.md file. It defines six intensity levels (lite, full, ultra, and three "wenyan" classical-Chinese variants), switchable with a slash command or natural language such as "talk like a caveman" or "less tokens." The instructions explicitly carve out exceptions: code blocks, commit messages, PR bodies, CLI/API names, and exact error strings are always left unabbreviated, and the compressed style is meant to switch off automatically for security warnings, irreversible-action confirmations, and any place where compression itself would create ambiguity.

2.2 What the installer sets up beyond the prompt

The published one-line installer does more than drop a SKILL.md file. For Claude Code specifically, by default it also:

- **Installs a session hook —** so caveman mode auto-activates from the first message without the user typing a slash command, and writes a small flag/log file each session.
- **Registers a token-savings stats feature —** which reads Claude Code's local session logs to compute lifetime tokens saved for a status-line badge.
- **Registers "caveman-shrink," an MCP proxy —** middleware that wraps other MCP servers (its own example wraps a filesystem server) and compresses their tool descriptions before Claude Code sees them.

A companion ecosystem exists under the same maintainer: caveman-code (a full terminal coding agent), cavemem (cross-agent memory), and a fine-tuned model — none required for the base skill, but discoverable from the same repository.

Source: JuliusBrussee/caveman — GitHub repository

3. Anthropic's Stated Position on Skill Security

Anthropic's own guidance on skills is directly relevant here. It identifies two headline risks for skills in general — prompt injection, which can manipulate Claude into unintended actions, and data exfiltration, caused by malicious package code or prompt-injected data leaks — and its explicit instruction is to install skills only from trusted sources.

Source: Anthropic Help Center — "Use skills in Claude"

Caveman is not distributed through Anthropic's official, security-reviewed plugin directory. By Anthropic's own framing, that places it outside the "trusted source" bar an enterprise should default to for anything touching confidential work, regardless of how benign the terse-writing behavior itself looks.

4. Security Analysis for a Highly Confidential Enterprise Setting

4.1 Prompt-layer trust — Low–Medium

The SKILL.md content only changes phrasing today. But because it is pulled from a third-party repository and re-loaded into Claude Code's persistent context every session, a future compromised commit, a maintainer account takeover, or a careless merge of a community pull request could add instructions well beyond "be terse" — the same risk class as any unreviewed CLAUDE.md or plugin. It should be pinned to a reviewed commit and re-diffed on update, not auto-pulled from a moving branch.

4.2 Installer / supply chain — High

The default install path is a curl|bash against the repository's main branch — not a signed release, tag, or pinned commit — with no checksum or signature verification at any step. On Claude Code specifically, two further defaults compound this: a hooks component is installed by fetching and executing a second remote script the same way, and the MCP proxy is registered via npx -y, which auto-accepts whatever version is currently published to the npm registry at install time. None of this is pinned, so what actually executes on a developer machine or CI runner is whatever the maintainer (or the npm registry) is serving that day.

4.3 MCP proxy interposition — Medium–High where MCP is used

"caveman-shrink" is designed to sit between Claude Code and any MCP server it wraps, rewriting the tool descriptions the agent relies on to decide how and when to call them. In an enterprise context, if that proxy wraps a server with access to confidential systems (file storage, ticketing, internal databases), it becomes a new node that can see, and in principle alter, that traffic — and even without any malicious intent, lossy compression of a tool's description can cause the agent to misunderstand a tool's contract and misuse it against sensitive data.

4.4 Local session-log parsing — Medium

The stats/badge feature reads Claude Code's own session logs locally to count saved tokens. Those logs can contain proprietary source, confidential business content, or secrets pasted during debugging. The project states this stays local with no telemetry calls — but a component that already has read access to full transcripts is a larger blast radius if a later auto-fetched update behaves differently than today's version.

4.5 Output-fidelity / compliance risk — Medium

The compression rules, including the safety carve-out, are instructions Claude is trusted to apply — not a hard, code-enforced gate. Under heavy compression (particularly "ultra" mode), the model can still occasionally compress a warning it should have left verbose, or drop a word that disambiguates the order of a multi-step operation — a failure mode the project's own documentation acknowledges as a known risk of the format. Separately, terse review or audit output is designed to strip the discursive detail (impact, rationale, references) that compliance-oriented processes typically expect to see recorded, even when the technical fix itself is accurate.

5. Tests Performed

Note on method: this environment has no outbound network access from its code-execution sandbox and no installed Claude Code runtime, so a live plugin install/run was not possible. Test 1 is a genuine static review of the maintainer's published, public installer script. Tests 2 and 3 are rule-application simulations — the skill's own documented ruleset (fetched verbatim from its SKILL.md) applied by hand to representative enterprise scenarios — and are labeled as such rather than presented as live product output.

Test 1 — Static review of the public installer (install.sh)

Method	Fetched and read the maintainer's published install.sh from the GitHub repository.
Finding 1	Installer runs curl|bash against the main branch — not a pinned tag, release, or commit hash — with no checksum or signature check anywhere in the script.
Finding 2	For Claude Code, "install hooks" and "register MCP proxy" are both ON by default (not opt-in). The hooks step performs a second, chained curl|bash against a different remote script.
Finding 3	The MCP proxy is registered with npx -y, which silently accepts whatever version of the package is currently published to npm — unpinned, re-resolved on every invocation.
Finding 4	An opt-in flag writes rule files directly into whatever directory the installer happens to be run from, which could land inside a proprietary repository if invoked there.
Finding 5	The same installer probes for and can configure 30+ other coding tools on the machine (Cursor, Windsurf, Copilot, etc.), meaning a single run can touch far more local configuration than "just Claude Code."
Verdict	The installer, not the terse-writing prompt, is the largest concrete source of security exposure in this project.

Test 2 — Auto-Clarity behavior on a destructive operation (simulation)

Scenario	"This migration drops the legacy_users column and the script says it backs up data first — should I run it?"
Spec-compliant behavior	Per the skill's own rules, caveman mode should switch off for this reply: state the irreversible action and the exact command in full, unambiguously, then resume terse style once the risk has been communicated clearly.
Plausible failure mode (documented, not observed live)	Because the safety carve-out is a judgment call the model is trusted to make rather than a hard, code-enforced gate, a heavily compressed ("ultra") reply under context pressure could still compress the warning itself, or drop the word that disambiguates step order — reintroducing the exact ambiguity risk the project's own documentation admits as a known limitation of the format.
Verdict	The safety carve-out is well designed on paper, but it is a soft, prompt-level control enforced by the model — not a hard platform gate. It should not be relied on as the sole safeguard for destructive operations; Claude Code's own confirmation prompts remain the actual control.

Test 3 — Compression fidelity on a security code-review use case (simulation)

Scenario	A SQL-injection finding during a PR review, compared across the skill's own three main intensity levels.

Baseline (verbose)	A full paragraph naming the vulnerable line, explaining the injection vector, describing likely impact, and recommending parameterized queries plus a follow-up re-review.
Full-caveman style	Compressed to a short line naming the file/line, the defect class, and the fix direction — matches the project's own documented "one-line PR comment" behavior.
Ultra-caveman style	Further abbreviated toward symbol/arrow notation for the same finding.
Retained at every level	File/line reference, defect class, and a concrete fix direction — the technical substance survives compression, consistent with the skill's design intent.
Dropped at full/ultra	The impact narrative, any standards reference (e.g., CWE/OWASP), and the reviewer's reasoning trail — exactly the material that SOC 2, ISO 27001, or PCI-DSS-style code-review evidence is typically expected to retain.
Verdict	Technically accurate but audit-thin. Adequate for a developer fixing the bug in the moment; weaker as a standalone compliance artifact unless paired with a fuller record kept elsewhere (ticket system, SAST tool output, etc.).

6. Risk Rating Summary

Concern	Likelihood	Impact	Overall
Prompt content drift or poisoning on a future repo update	Low	Medium	**Low–Med**
Installer supply chain (unpinned curl|bash, chained scripts, npx -y)	Medium	High	**High**
MCP proxy (caveman-shrink) interposed between agent and data	Low–Med	High if used	**Med–High**
Local session-log parsing by the stats/badge feature	Low	Medium	**Medium**
Compression dropping audit-relevant detail from reviews	Medium	Medium	**Medium**

7. Recommendations for Enterprise / Confidential Deployment

- **Treat it like any unreviewed third-party plugin —** fork and pin to a specific, reviewed commit; distribute via an internal mirror rather than curl|bash to a live main branch; re-diff before every update, consistent with Anthropic's own "trusted sources only" guidance for skills.

- **Install with the minimal option —** skip the hooks component and the MCP proxy registration unless each has separately been security-reviewed; these are the two components with the broadest local system and data access.
- **Never treat terse output as the compliance record —** keep or generate a fuller companion artifact (ticket, SAST finding, review note) for anything that needs an audit trail; don't rely on one-line PR comments alone for security sign-off.
- **Keep destructive-operation guardrails independent of the skill —** the Auto-Clarity behavior is a helpful prompt-level habit, not a substitute for Claude Code's own confirmation prompts and your existing change-control process.
- **Review any MCP proxy before it touches confidential data —** if MCP servers are in use, avoid wrapping ones connected to sensitive systems with a third-party compressing proxy until that proxy itself has been reviewed like any other MCP server.

8. Conclusion

The core idea behind the caveman skill — make Claude's spoken output terser while leaving code, commands, and errors untouched — does not, by itself, weaken Claude Code's safety behavior in a way that is unique to this skill. Nearly all of the meaningful additional risk in a highly confidential enterprise setting comes from how it is distributed and installed by default: unpinned remote scripts, a default-on MCP proxy sitting in front of other data sources, and a local component that parses session transcripts. It should be evaluated, pinned, and gated the same way any other unreviewed third-party Claude Code plugin would be — not adopted as "just a style tweak."